

DroneArjuna

Ground Control System for Heterogeneous Drone Fleets
Project Report — Features, Functionality & System Architecture

RESTRICTED / CONFIDENTIAL — ACS Technologies Limited

Prepared: September 2026

1. Project Overview

DroneArjuna is a military-grade Ground Control System (GCS) developed by ACS Technologies Limited for command, control, and monitoring of heterogeneous drone fleets. It provides real-time MAVLink telemetry, multi-drone fleet management, 2D mission planning, live video/payload feeds, AI-assisted analytics, and a structured knowledge base of drone platforms, payloads, and threat systems. It is positioned as a purpose-built, decisively superior alternative to general-purpose tools such as QGroundControl, with additional support for naval/HF-linked drone operations.

The system follows a **microservices-oriented modular monolith** architecture, organized into five functional modules — **Drone Control**, **Drone Master**, **Drone Inventory**, **Drone Flight**, and **Drone Analyst** — each with dedicated API routers, services, and (where implemented) frontend workspaces.

2. Technology Stack

2.1 Backend

Layer	Technology
Language / Runtime	Python 3.11
Web Framework	FastAPI 0.111, Uvicorn (uvloop on Linux/Docker)
Data Validation	Pydantic 2 / pydantic-settings
ORM / Migrations	SQLAlchemy 2.0 (async) + asyncpg, Alembic
Caching / Messaging	Redis (aioredis), RabbitMQ (aio-pika)
Auth & Security	python-jose (JWT), passlib[bcrypt]
Drone Telemetry / Control	pymavlink, pyserial, pynmea2
Computer Vision	opencv-contrib-python-headless (payload camera, TrackerCSRT)
Geospatial	shapely, pyproj (geofencing, airspace)
Fleet Optimization	OR-Tools (classical solver); optional Qiskit/QAOA quantum path
Object Storage Client	boto3 (MinIO/S3-compatible)
Search	elasticsearch[async] 8.12 (knowledge-base search)
Email	aioSMTPlib
Logging / Serialization	structlog, orjson
Testing	pytest, pytest-asyncio, anyio, aiosqlite (in-memory CI DB)

2.2 Frontend

Layer	Technology
Framework	React 18 + TypeScript
Build Tool	Vite 5
State Management	Zustand
HTTP Client	Axios
Mapping (2D)	Leaflet + react-leaflet
Charts	Recharts

Layer	Technology
Icons	lucide-react
Styling	Tailwind CSS
Date Handling	date-fns

Note: 3D mapping (Cesium) is referenced in project documentation as a planned capability but is not present in the current frontend package.json.

2.3 Data Stores & Infrastructure

Component	Technology	Purpose
Relational DB	PostgreSQL 15 + PostGIS	Core entities, spatial queries
Time-Series DB	TimescaleDB	High-frequency telemetry hypertables
Cache / Session	Redis 7	Caching, session state
Message Broker	RabbitMQ 4.3	Event bus (AMQP) for system events
Object Storage	MinIO	Video, imagery, analysis reports, backups
Search Engine	Elasticsearch 8.12	Full-text knowledge-base search
Dev SMTP	MailHog	Local email testing (password reset, notifications)
Admin UI	pgAdmin 4	Database administration

3. System Architecture

DroneArjuna is architected as a **modular monolith**: a single FastAPI backend process hosting five independently namespaced modules, backed by a polyglot persistence layer (PostgreSQL/PostGIS for structured entities, TimescaleDB for time-series telemetry) and supporting infrastructure services (Redis, RabbitMQ, MinIO, Elasticsearch). This design gives clear module boundaries for future extraction into true microservices while avoiding the operational overhead of distributed systems at the current project stage.

3.1 High-Level Component Diagram (textual)

React SPA (Vite dev server, port 3000) → Axios/WebSocket calls to → FastAPI backend (port 8000) → SQLAlchemy async sessions → PostgreSQL+PostGIS (port 15432) and TimescaleDB (port 5433). Backend also connects to Redis (cache), RabbitMQ (event bus), MinIO (object storage), and Elasticsearch (search indexing). A pymavlink-based MAVLink Manager bridges the backend to physical/simulated drones over UDP/TCP/Serial, broadcasting telemetry to the frontend via WebSocket at 10 Hz and rebroadcasting to external GCS clients over UDP 14560. A separate HF/NMEA feed (UDP 10110) supports naval vessel position integration.

3.2 Backend Module Boundaries

Module	Responsibility	Status
Drone Control	Live MAVLink connectivity, telemetry streaming, command issuance, geofencing, payload camera/tracking	Complete
Drone Master	Drone type/instance registry, naval vessel registry, payload types, config templates	Complete
Drone Inventory	Drone/payload/threat-system catalog, cross-reference links, knowledge-base search	Backend complete; no frontend
Drone Flight	Mission planning, waypoint management, validation, fleet assignment, survey-grid generation	Complete

Module	Responsibility	Status
Drone Analyst	AI analysis job lifecycle, artifact storage, model/results reporting	Job pipeline complete; inference not yet implemented

3.3 Cross-Cutting Concerns

- **Authentication & Authorization:** JWT-based auth with a 5-role RBAC model enforced via FastAPI dependencies on individual routes.
- **Event Bus:** RabbitMQ-based event emission for telemetry, connect/disconnect, mission status changes, health alerts, and geofence breaches.
- **Connection Health Monitoring:** WebSocket auto-reconnect (up to 5 retries, exponential backoff), 30-second heartbeat, 0–100 health scoring on the frontend.
- **Audit / Event Logging:** Frontend event log store with export to JSON/CSV and 5,000-event retention.
- **Backup & Restore:** Scheduled database dumps with integrity monitoring and MinIO-backed restore-on-empty-volume for both PostgreSQL and TimescaleDB.
- **Naval/HF Integration:** NMEA GGA feed ingestion and HF link adapter for ship-borne drone operations with dynamic return-to-vessel behavior.

4. Features & Functionality by Module

4.1 Drone Control Module

The most mature module — provides genuine (non-mocked) MAVLink integration for real-time command and telemetry.

- Multi-transport MAVLink connectivity: UDP, TCP, Serial, HF-Serial, HF-TCP, with async heartbeat and telemetry read loops.
- Serial port auto-scan / auto-connect for physical hardware.
- Command controller for issuing flight commands to connected drones.
- Real-time telemetry processing, health monitoring, and proximity/collision monitoring.
- Telemetry data recording into TimescaleDB hypertables for historical playback and battery logging.
- Mission simulator supporting virtual (simulated) drones for testing without hardware.
- MAVLink broadcaster — rebroadcasts live telemetry to external GCS clients over UDP.
- Payload camera integration: RTSP/webcam capture with OpenCV-based object tracking (TrackerCSRT).
- Per-drone and fleet-wide geofence definition and retrieval.
- WebSocket streaming for live telemetry (10 Hz) and a separate camera video stream.
- Frontend: Fly workspace (live map, instrument HUD, command panel) and Monitor workspace (gauge dashboard, telemetry charts, system log).

4.2 Drone Master Module

Registry and configuration management for all fleet assets.

- Full CRUD for drone types (manufacturer, model, size class, mission type, VTOL capability, performance specs, autopilot type).
- Full CRUD for drone instances (call sign, serial number, MAVLink system ID, status, last-seen, flight hours) with status patching and stale-record cleanup.

- Naval vessel registry with live position updates and drone assignment/unassignment.
- Payload type registry (sensor, combat, comms, other categories) with type-specific attributes.
- Reusable configuration templates that can be applied directly to a drone instance.
- Frontend: Settings workspace (drone type/instance managers, payload manager, vessel manager, user manager) and Fleet workspace (fleet overview, drone cards, connect modal).

4.3 Drone Inventory Module

Houses the threat-system knowledge base referenced in prior project work, along with the broader drone/payload catalog.

- Drone and payload catalog browsing, side-by-side comparison, and search.
- Full CRUD for threat systems (category: UAV/RADAR/SAM/EW; manufacturer, country, range, altitude, speed, RCS, countermeasures, classification) — 25 backend tests covering this API.
- Cross-reference link tables: drone-payload compatibility, drone-threat exposure level (LOW/MEDIUM/HIGH/CRITICAL), and payload-threat effectiveness rating.
- Knowledge-base endpoints for capability lookups (by drone/payload), threat mitigation and vulnerability analysis, and full-text KB search backed by Elasticsearch.
- Analytics endpoints for KB health and per-entity analytics.
- Gap: this module currently has no dedicated frontend workspace or API client, despite roughly 35 backend endpoints.

4.4 Drone Flight Module

Mission planning and fleet-level flight coordination.

- Mission CRUD with status lifecycle (planning → approved → executing → completed → aborted).
- Waypoint management with per-waypoint actions (loiter, photo, survey, payload trigger).
- Mission validation and waypoint upload to connected drones; live-sync of missions to active drone instances.
- Mission simulation for pre-flight verification.
- Fleet assignment for multi-drone missions, including an optional quantum-computing assignment path (OR-Tools classical solver by default; Qiskit/QAOA optional, degrades gracefully if unavailable).
- Survey-grid generation for area-coverage missions.
- Geofencing and airspace enforcement (shapely/pyproj-based) with backend-enforced auto-return-to-launch on geofence breach.
- Support for both fixed home points and dynamic vessel-relative home points (naval operations).
- Frontend: Plan workspace with Leaflet-based map canvas, mission editor, fleet assignment modal, and live operations panel.

4.5 Drone Analyst Module

AI-assisted analysis job orchestration.

- Job lifecycle state machine (queued → running → done/failed) with parameter and result storage.
- Artifact storage in MinIO (images, video, PDF reports) with per-job artifact CRUD.
- Job cancellation, results retrieval, model listing, and per-mission statistics/series endpoints.
- Job types defined for object detection, change detection, video analysis, and telemetry reporting.
- Gap: the AI inference pipeline itself is not yet implemented — job execution currently returns a placeholder result rather than running an object-detection model. No dedicated frontend workspace exists for this module yet.

4.6 Authentication, Authorization & User Management

- JWT-based login/refresh token flow with bcrypt password hashing.
- Password lifecycle: forced password change on first login, self-service forgot/reset password (token + expiry), and admin-triggered password reset.
- Self-service access request workflow: unauthenticated users submit a request; an admin accepts (auto-registers with a temporary password and triggers email) or rejects.
- Role-Based Access Control with five roles — Admin, Mission Commander, Flight Controller, Intelligence Analyst, Viewer — enforced per-route via dependency injection, supporting both exact-role and minimum-role-level checks.

5. API Overview

The backend exposes a REST API (with two WebSocket streams) organized by module, with interactive documentation auto-generated by FastAPI at runtime.

5.1 Authentication & User Management

Covers login and token refresh, retrieving the current user's profile, first-login password setup, admin-driven user registration and password resets, self-service forgot/reset-password, and the full self-service access-request workflow (submission, listing, accept/reject, and resend of the invitation email).

5.2 Drone Control

Covers discovering available serial ports and auto-connecting to hardware, checking connection status, connecting to and disconnecting from individual drones, reading and updating per-drone geofences, sending flight commands, retrieving live and historical telemetry (including gauge snapshots and battery logs), starting and stopping simulated drones, and two live WebSocket streams for telemetry and camera video.

5.3 Drone Master

Covers full management of drone types, drone instances (including status updates and cleanup of stale records), naval vessels (including live position updates and drone assignment/unassignment), payload types, and reusable configuration templates that can be applied to a drone instance.

5.4 Drone Inventory

Covers browsing and comparing the drone/payload catalog with search, full management of threat-system records, the cross-reference link tables between drones, payloads, and threats, and the knowledge-base endpoints for capability lookups, threat mitigation/vulnerability analysis, full-text search, and analytics.

5.5 Drone Flight

Covers mission creation, retrieval, updates, and deletion; mission validation and waypoint upload to a drone; live mission sync to an active drone instance; mission simulation; fleet-wide mission assignment; and survey-grid generation for area-coverage missions.

5.6 Drone Analyst

Covers submitting and tracking analysis jobs, managing job artifacts, cancelling jobs, retrieving results, listing available models, and per-mission analysis statistics and time series.

5.7 System

Covers a basic health check endpoint and a WebSocket stream for system-wide events.

6. Database Schema

6.1 Primary Database (PostgreSQL + PostGIS)

Entity	Key Fields / Purpose
User	username, email, hashed_password, role, must_change_password, reset token/expiry
AccessRequest	Self-service registration workflow (pending/approved/rejected), temp_password
DroneType	manufacturer, model, size_class, mission_type, VTOL, performance specs, autopilot_type
DroneInstance	call_sign, serial_number, mavlink_system_id, status, last_seen, flight hours, home_vessel_id, payload_type_id
DroneConfigTemplate	Reusable JSON settings blob keyed to a DroneType
Mission	type, status, geofence (GeoJSON), home_point_type (fixed/dynamic_vessel), enforceairspace
Waypoint	sequence, lat/lon/alt, action (loiter/photo/survey/payload_trigger)
NavalVessel	vessel_id, live lat/lon/heading/speed, sea_state, deck_status, HF modem config
PayloadType	category (sensor/combat/comms/other), weight/voltage/current, gimbal and category-specific fields
ThreatSystem	category (UAV/RADAR/SAM/EW), manufacturer, country, range/altitude/speed, RCS, countermeasures, classification
AnalysisJob	job_type, status state machine, params/result JSON
JobArtifact	MinIO object_key, kind (image/video/report_pdf)
DronePayloadLink / DroneThreatLink / PayloadThreatLink	Cross-reference link tables with exposure/effectiveness ratings, unique constraints, cascade FKs

6.2 Time-Series Database (TimescaleDB)

Entity	Purpose
TelemetryFrame	Current-state snapshot, one row per drone (position, attitude, velocity, battery, GPS, mode, RSSI, CPU load)
TelemetryHistory	Append-only flight-path log for historical/3D replay (1-day retention)
TelemetryGauge	Dashboard gauge snapshot (battery, altitude, speed, GPS satellites, RSSI, CPU)
BatterySnapshot	Periodic battery log, sampled every 2 minutes during active missions
SystemMetric	Backend health metrics (CPU, memory, queue depth)

Schema managed via 17 Alembic migrations (backend/alembic/versions) for the primary database; TimescaleDB hypertables are created via raw SQL at startup, outside the Alembic chain.

7. Testing & Quality Assurance

- Backend: pytest + pytest-asyncio + anyio, run against an in-memory SQLite database for CI speed and isolation.

- 63 backend test files covering all five modules plus infrastructure (auth, RBAC, threat system API — 25 tests, inventory knowledge base, MAVLink broadcaster, payload camera, geofencing, missions, fleet assignment, analyst jobs, backup/restore, event bus, search, object storage, HF feed, vessel integration, and unauthenticated-access checks).
- Coverage reports generated per test run (coverage.json, HTML coverage, per-module manifests).
- Continuous Integration (GitHub Actions): backend pytest suite, frontend TypeScript type-check + build, and a Docker build-verification job, triggered on pushes/PRs to main and develop.
- Frontend unit testing is not yet configured in the current package.json (no test runner/framework present).

8. Deployment & Infrastructure

The full stack is defined in a single Docker Compose file comprising nine services on a shared bridge network:

Service	Image / Build	Purpose
postgres	postgis/postgis:15-3.3	Primary relational database
timescale	timescale/timescaledb:latest-pg15	Time-series telemetry database
pgadmin	dpage/pgadmin4:8.14	Database administration UI
redis	redis:7-alpine	Cache / session store
rabbitmq	rabbitmq:4.3.3-management-alpine	Event bus (AMQP) + management UI
minio	quay.io/minio/minio:latest	Object storage (video, imagery, reports, backups)
elasticsearch	elasticsearch:8.12.0	Knowledge-base full-text search
mailhog	mailhog/mailhog:latest	Development SMTP inbox
backend	./backend/Dockerfile (FastAPI)	Application API, MAVLink bridge, WebSocket streaming
frontend	./frontend/Dockerfile (Vite)	React single-page application

The backend container runs in privileged mode to allow USB serial passthrough for physical drone hardware, and exposes MAVLink SITL (14550/udp), telemetry broadcast (14560/udp), and vessel NMEA (10110/udp) ports alongside its HTTP API (8000). Persistent named volumes back all stateful services. CI currently builds and tests the stack but does not deploy it — there is no Kubernetes/Terraform configuration; Docker Compose is the sole deployment target today. Supporting PowerShell scripts (start/stop, autostart, COM-port bridge service) manage local Windows-based development and hardware-connected deployments.

9. Known Gaps & Documentation Notes

The following items were identified during codebase review and are worth tracking for the next development phase:

- **AI inference not implemented:** the Drone Analyst job pipeline is functionally complete, but object-detection/AI inference (e.g., YOLOv8, advertised in project documentation) currently returns a placeholder result rather than real model output.
- **Missing frontend coverage:** Drone Inventory (~35 endpoints, including the threat-system knowledge base) and Drone Analyst (~13 endpoints) have no corresponding frontend workspace or API client yet.
- **Frontend test coverage:** no test framework is currently configured in the frontend build.
- **Documentation drift:** some README endpoint paths (e.g., login route, docs URL) and the RBAC role table in project architecture notes are slightly out of date relative to the current codebase; the live code should be treated as authoritative.